

High Performance Computing

Lecture 13: Accelerators

Beakal Gizachew Assefa

So Far

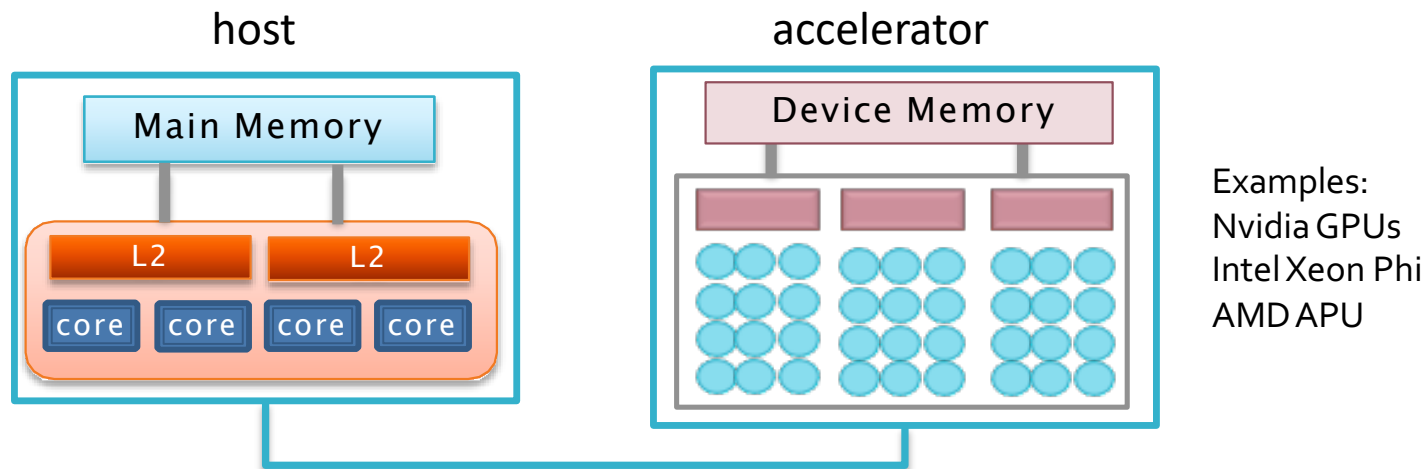
We have covered

- Parallel Architectures
- Performance Characterization
 - Speedup, Efficiency, Modeling, Communication Cost
- Shared Memory Programming Models
 - Pthreads
 - OpenMP
- Message Passing Programming Models
 - MPI
- Task vs Data Parallelism
- Next
 - Parallel Programming Accelerators

Accelerator--based Systems

⦿ Accelerators

- › Typically attached to a host (a multicore CPU)
- › Massively parallel single chip processor
- › Low power cores: trade off single thread performance
- › Offload compute intensive parts of the program to the accelerator
- › There is a bus connecting accelerator to the host



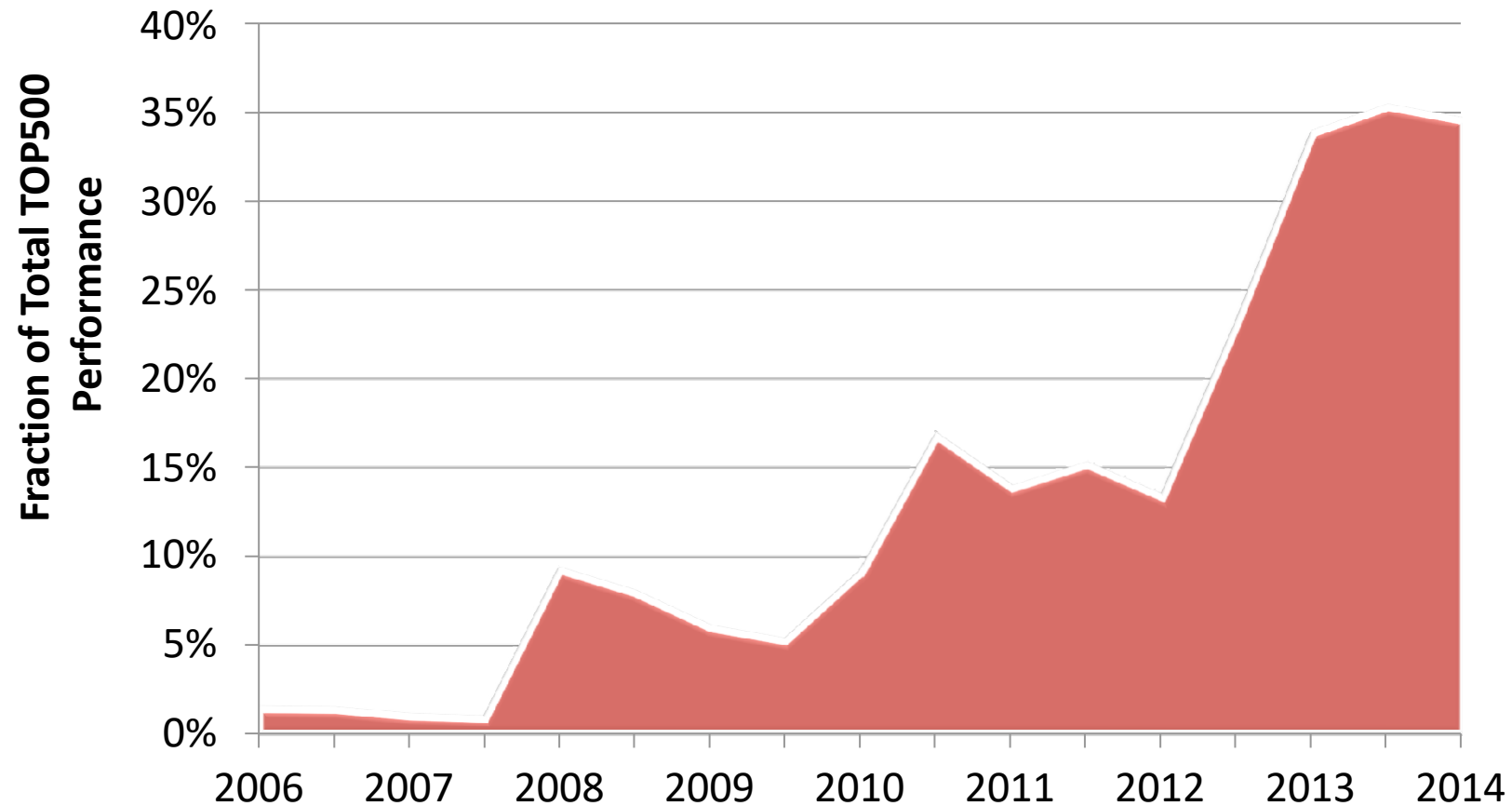
Top 500

- #1 Supercomputer in the world Tianhe-1 is a hybrid design with two types of accelerators
- 14,336 Intel Xeon processors and
- 7,168 NVIDIA Tesla GPUs



Tianhe-2 in China Peak 54 Petaflops
Sustained ~34 Petaflops

Performance Share of Accelerators in Top 500



HPC facility on our campus has 18 GPUs (accelerators).

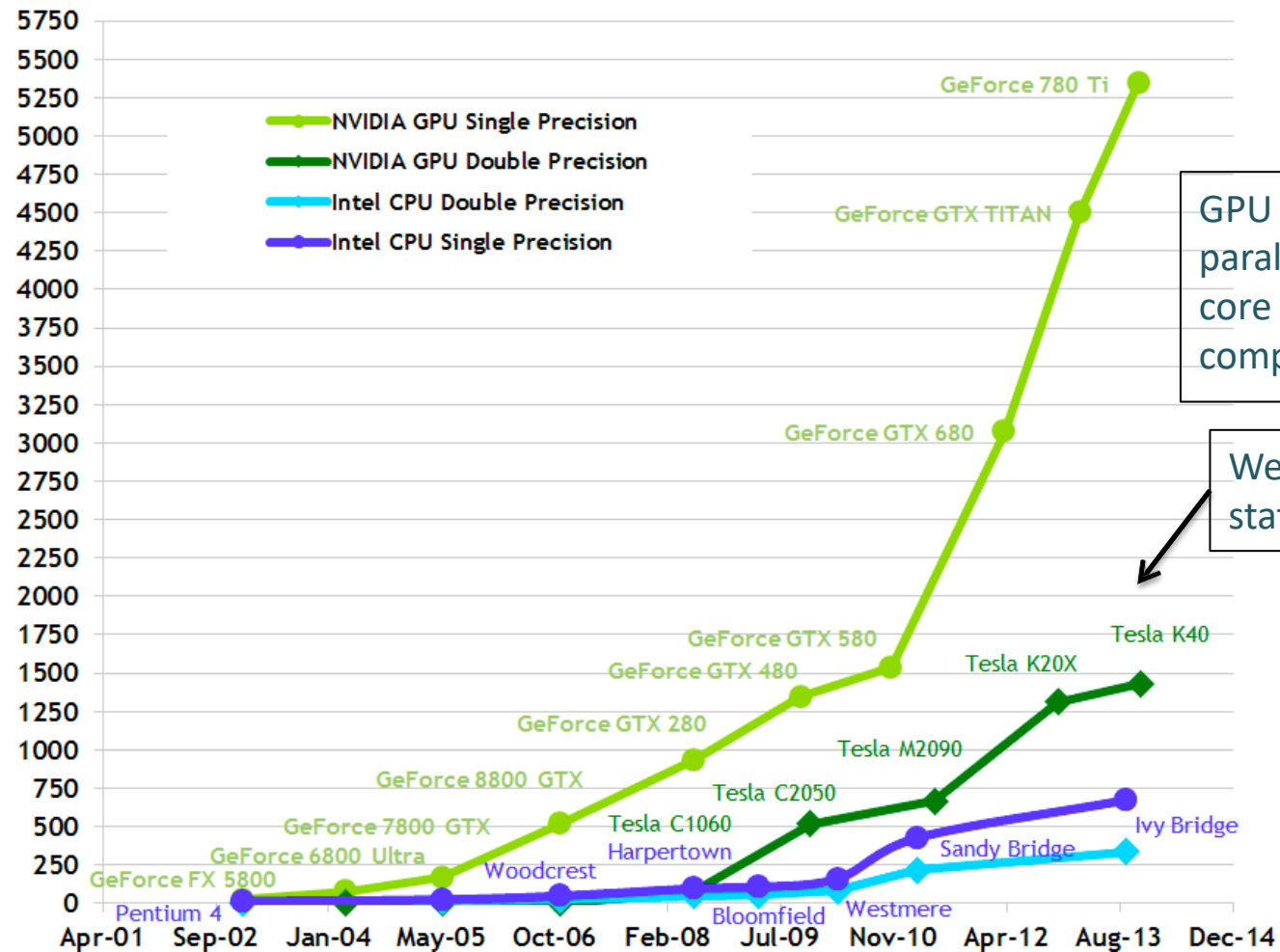
GPU Architectures

- In this class, we will focus on GPU accelerators
- Processing is highly data--parallel
 - GPUs are highly multithreaded
 - **Use thread switching to hide memory latency**
 - Graphics memory is wide and high--bandwidth



Computational Performance

Theoretical GFLOP/s



GPU has evolved into a highly parallel, multithreaded, many core processor with huge computational horsepower

We will be using Tesla K40—a state-of-the-art GPU

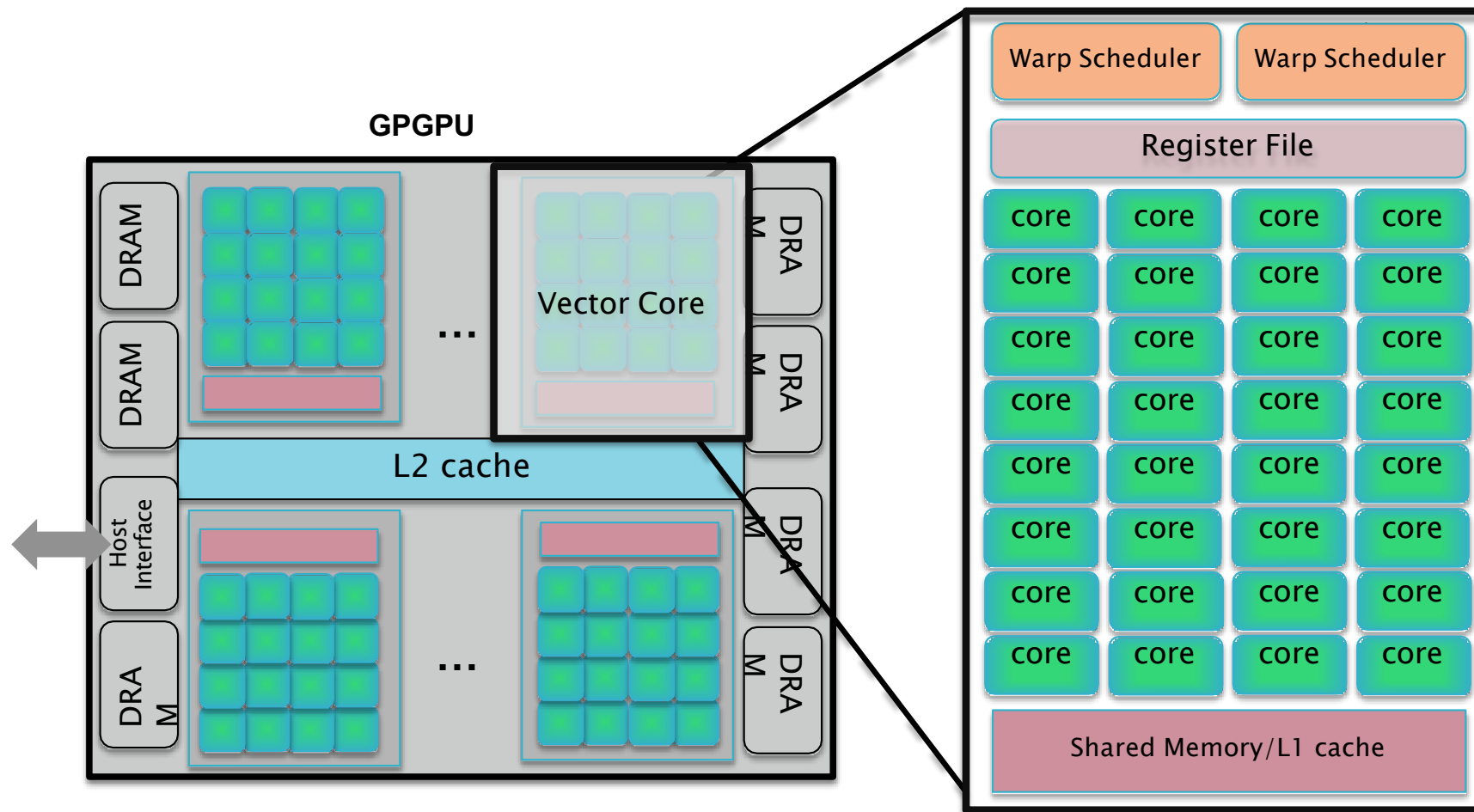
Img. Src Nvidia

CPU vs GPU



- CPU is optimized for latency--oriented applications
 - Individual cores are complex and powerful
- GPU is optimized for throughput--oriented applications
 - Cores are weak but together they devote more transistors to data processing
 - Tesla K40 has 2880 CUDA cores

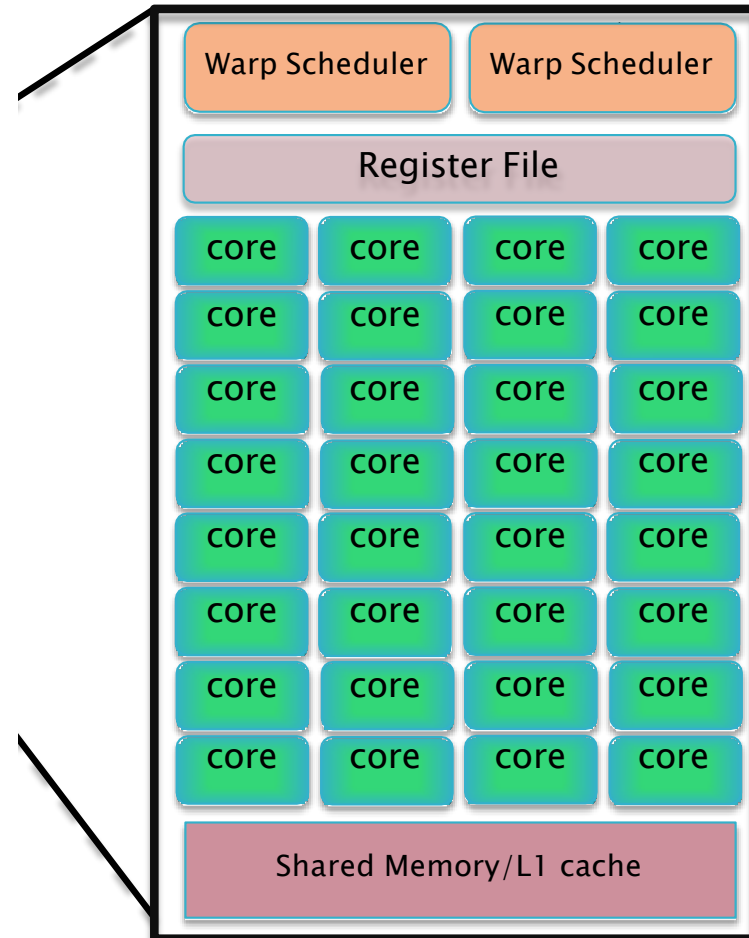
GPU Architecture



- A vector core (or stream multiprocessor) contains a large number of lightweight cores
- A GPU device has a number of vector cores (~10s)

GPU Architecture

- A stream multiprocessor (SM)
 - contains a large number of lightweight cores (CUDA cores)
- All the cores in a SM
 - Accesses to a shared memory and L1 cache
 - Shares shared memory (a.k.a) local storage is a software managed cache!
 - There is a large register file

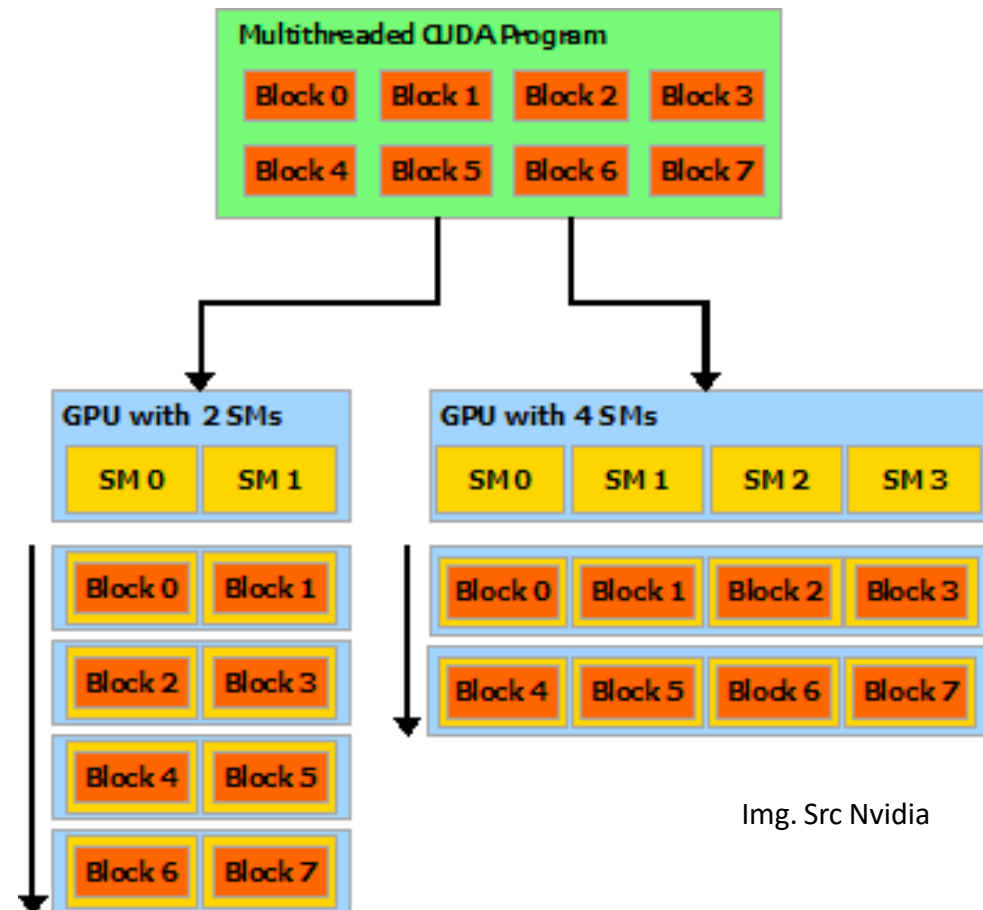


Execution Model

- GPUs use a SIMT model
 - Single Instruction Multiple Threads
- Combines SIMD execution within a block (on an Stream Multiprocessor) with SPMD execution across Blocks
 - Individual instruction streams for each thread are grouped together for SIMD execution on hardware
 - For example Nvidia groups 32 CUDA threads into a group (warp)
- GPU hardware handles thread management, not applications or OS

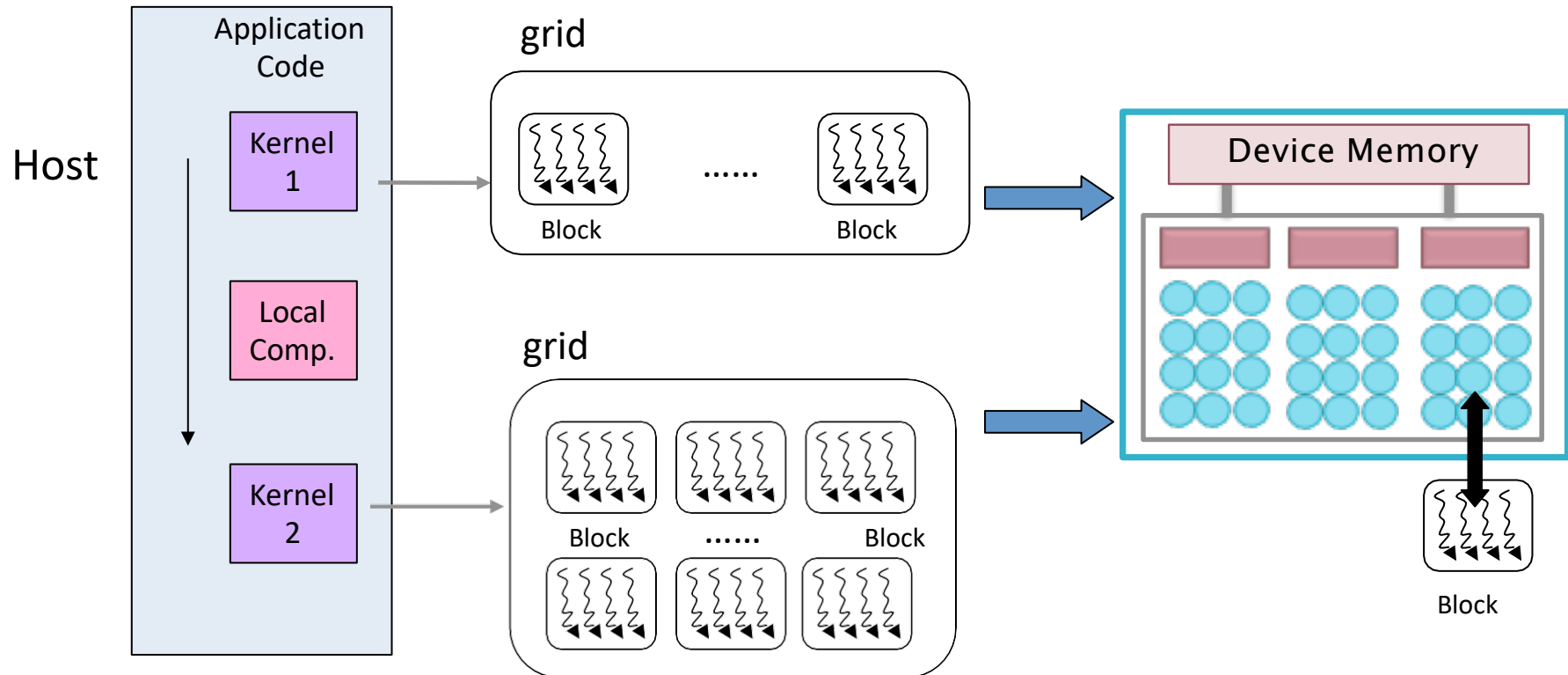
Thread Blocks

- A multithreaded program is partitioned into **blocks of threads** that **execute independently** from each other
- This is different than threading models we learnt
 - There is a hierarchy of threads
 - Threads in a block are peers, can communicate with each other
 - Threads across blocks execute independently



Img. Src Nvidia

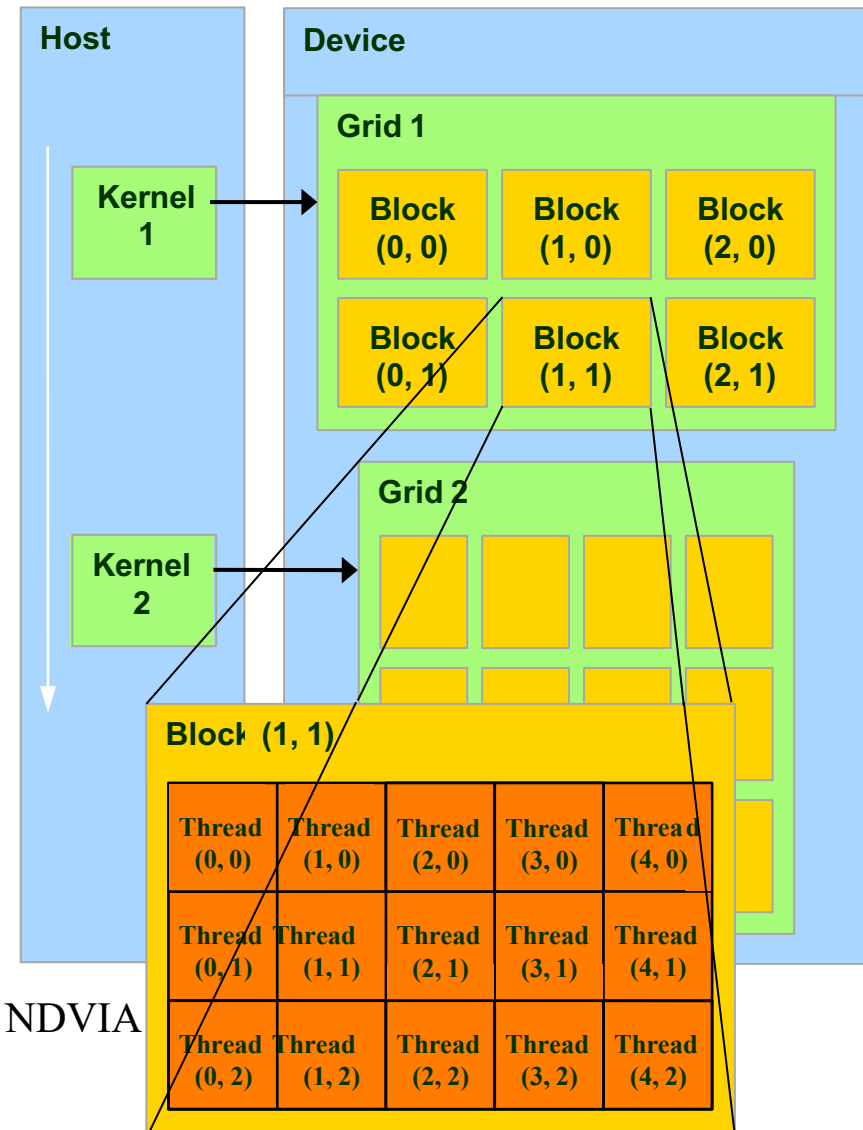
CUDA Programming Model



- Designed by Nvidia
- An extension of C/C++/Fortran
- A data parallel function
 - called “**kernel**”
 - offloaded to the GPU
- A kernel
 - Generates 1000s of threads
 - Threads are tiny, computes couple data elements
 - Organized as “**blocks**”

CUDA Thread Blocks and Grids

- A kernel is executed as a **grid of thread blocks**
 - All threads share data memory space
- A **thread block** is a batch of threads that can **cooperate** with each other by:
 - Synchronizing their execution
 - Efficiently sharing data through a low latency **shared memory**
- Two threads from two different blocks cannot cooperate



Courtesy: NDVIA

Compiling CUDA

- Any source file containing CUDA language code must be compiled with `nvcc`
 - It separates code running on the host from the code running on the GPU
 - Creates PTX (Parallel Thread eXecution) and CPU code
 - PTX is the assembly code for Nvidia GPUs
 - Then PTX code is compiled into device-specific binary object
- Device Emulation Mode
 - `nvcc -deviceemu`
 - Runs on the host using the CUDA runtime
 - Useful for debugging
 - Executes sequentially

Our First CUDA Program

- Our first CUDA Program: Increment Array
 - Simple CUDA Kernel
 - Kernel Launch and Kernel Parameters
 - Host Memory Management
 - Device Memory Allocation
 - Host-Device Memory Copy

Increment Array

```
void increment_Array_On_Host(float* A, int N)
{
    int i;
    for (i=0; i< N; i++)
        A[i] = A[i] + 1.f;
}
```

On Host

```
__global__ void increment_On_Device(float *A, int N)
{
    int idx = blockIdx.x*blockDim.x + threadIdx.x;

    if (idx<N)
        A[idx] = A[idx]+1.0f;
}
```

On GPU

Kernel Launch

Function Call:

```
increment_Array_On_Host(A, N);
```

```
void increment_Array_On_Host(float* A, int N)
{
    int i;
    for (i=0; i< N; i++)
        A[i] = A[i] + 1.f;
}
```

On Host

Kernel Call:

```
increment_Array_On_Device<<<nBlocks, blockSize>>> (A_d, N);
```

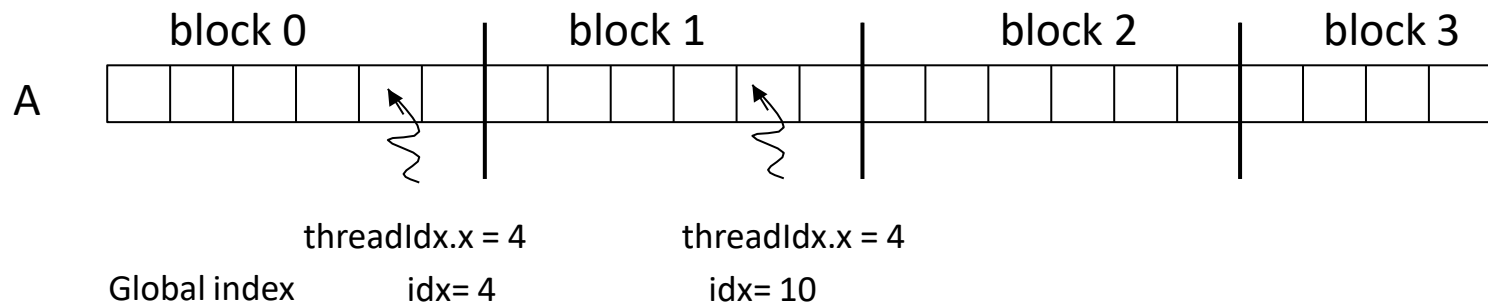
```
__global__ void increment_On_Device(float *A, int N)
{
    int idx = blockIdx.x*blockDim.x + threadIdx.x;

    if (idx<N)
        A[idx] = A[idx]+1.0f;
}
```

On GPU

Thread Assignment

- Each thread increments one element in A
- A thread uses *block Id*, *block size* and *thread Id* parameters to determine its workload.
- `__global__` keyword precedes the kernel declarations

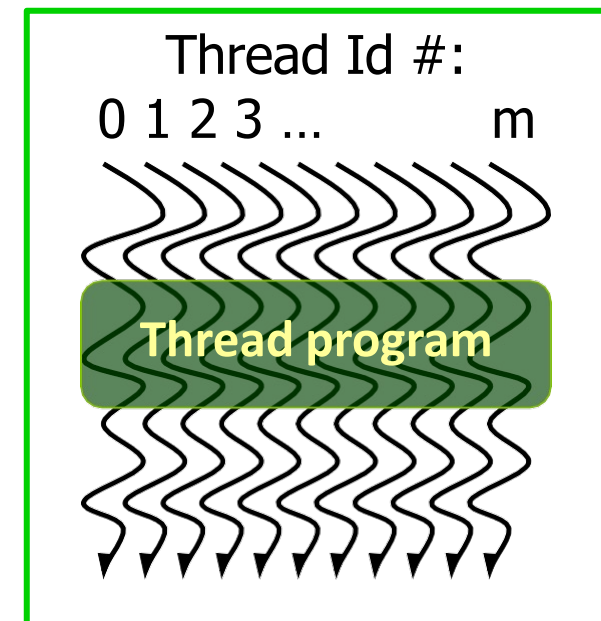


```
__global__ void increment_On_Device(float *A, int N)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N)
        A[idx] = A[idx] + 1.f;
}
```

CUDA Thread Block Overview

- All threads in a block execute the same kernel program (SPMD)
- Programmer declares block:
 - Block size 1 to **512** concurrent threads
 - Block shape 1D, 2D, or 3D
 - Block dimensions in threads
- Threads have **thread id** numbers within block
 - Thread program uses **thread id** to select work and address shared data
- Threads in the same block share data and synchronize while doing their share of the work
- Threads in different blocks cannot cooperate
 - Each block can execute in any order relative to other blocks!

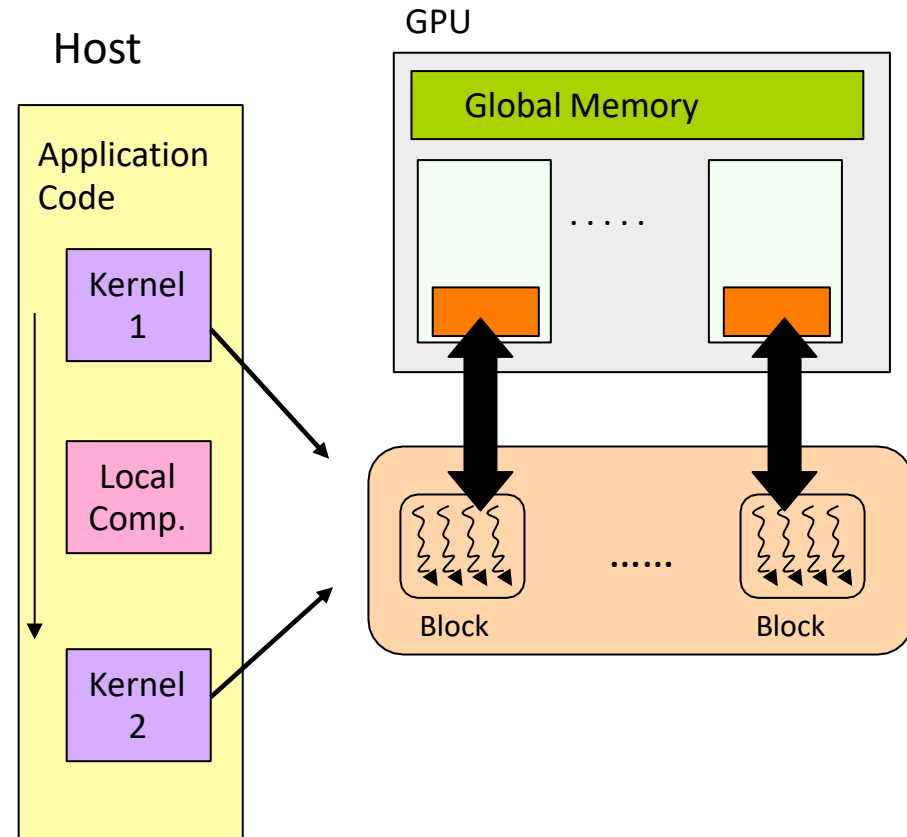
CUDA Thread Block



Courtesy: John Nickolls,
NVIDIA

Managing Memory

- *Increment Array* Kernel is offloaded to the GPU
- Array **A** resides on the main memory NOT in the device memory.
 - Allocate storage on the device memory
 - Copy data over the device before kernel launch



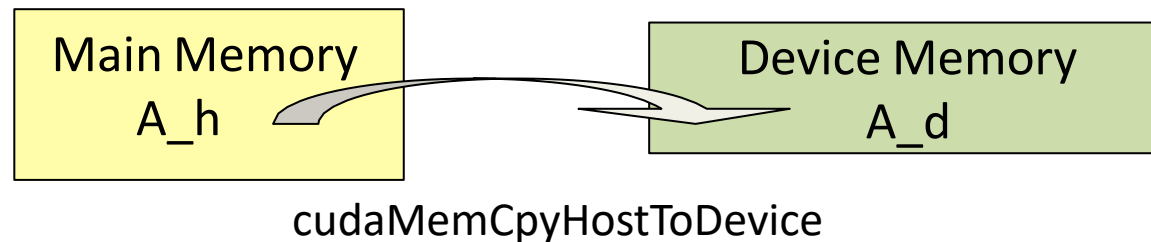
Managing Memory on the Host

```
float *A_h, *B_h;      // pointers to host memory
float *A_d;

//Allocate memory on the device for A
cudaMalloc((void **) &A_d, size);

for (i=0; i<N; i++)
    A_h[i] = (float)i;  // init host data

//Copy host array to the device array
cudaMemcpy(a_d, a_h, sizeof(float)*N, cudaMemcpyHostToDevice);
```



Kernel Launch on the Host

```
//set block size
int bSize = 64;

//compute number of blocks
int nBlocks = N/bSize + (N%bSize == 0 ? 0 : 1);

//kernel launch
increment_on_Device <<< nBlocks, bSize >>> (A_d, N);

//synchronize to make sure kernel is complete
cudaThreadSynchronize();
```

- Host asynchronously launches a kernel on GPU
 - Immediately returns from the kernel launch
 - Host can perform another computation on CPU
 - Need to call kernel synchronization before retrieving any results from the GPU

Copy Back the Result

```
//set block size
int bSize = 64;

//compute number of blocks
int nBlocks = N/bSize + (N%bSize == 0 ? 0 : 1);

//kernel launch
increment_on_Device <<< nBlocks, bSize >>> (A_d, N);

//synchronize to make sure kernel is complete
cudaThreadSynchronize();

// Retrieve result from device and store in B_h
cudaMemcpy(B_h, A_d, sizeof(float)*N, cudaMemcpyDeviceToHost);
```


Clean up

```
//set block size
int bSize = 64;

//compute number of blocks
int nBlocks = N/bSize + (N%bSize == 0 ? 0 : 1);

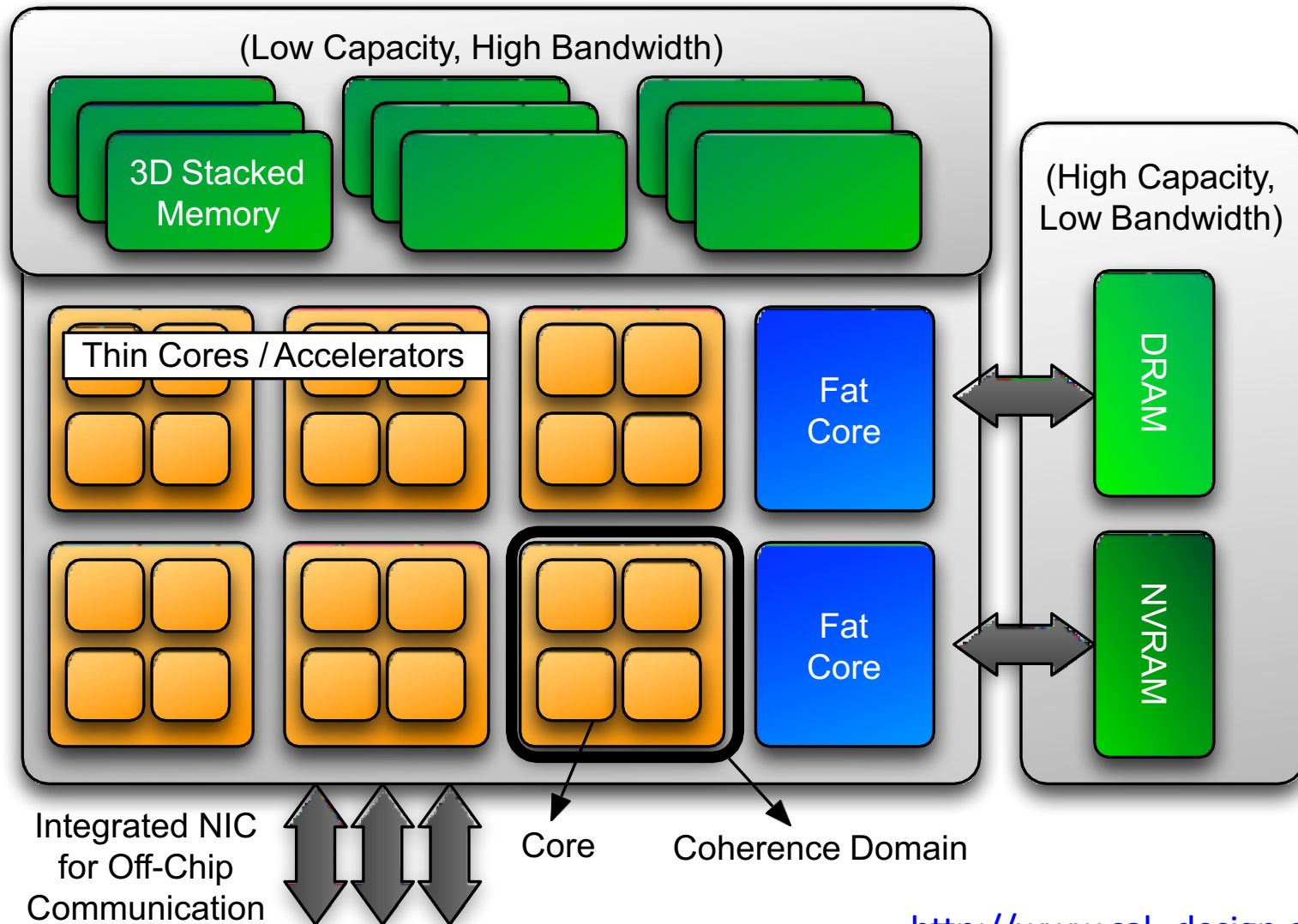
//kernel launch
increment_on_Device <<< nBlocks, bSize >>> (A_d, N);

//synchronize to make sure kernel is complete
cudaThreadSynchronize();

// Retrieve result from device and store in B_h
cudaMemcpy(B_h, A_d, sizeof(float)*N, cudaMemcpyDeviceToHost);

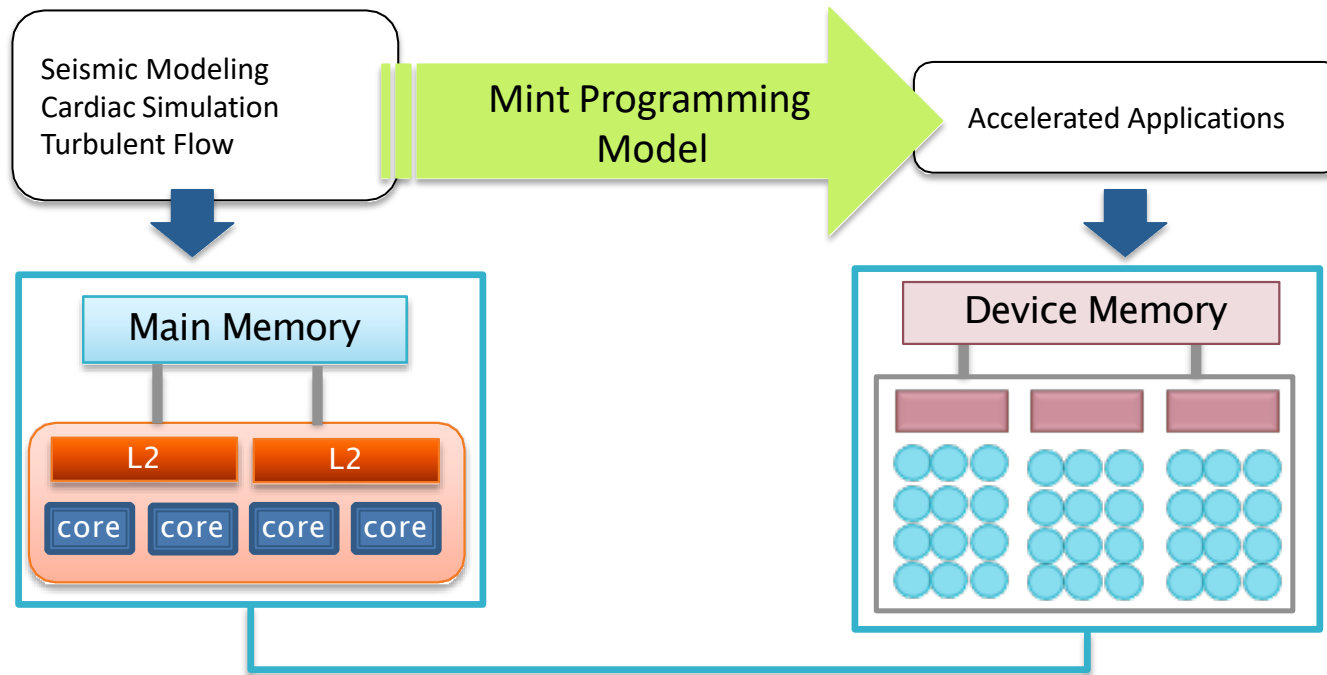
// cleanup
free(A_h);
free(B_h);
cudaFree(A_d);
```

Abstract Machine Model for Exascale (2022)



<http://www.cal--design.org/>

Mint Programming Model



- Abstracts away the programmer's view of the hardware
- Targets heterogeneous systems
- Based on a modest number of compiler directives
 - `#pragma mint for`
 - Incremental parallelization

GPU Computing Applications

Libraries and Middleware

CUFFT CUBLAS CURAND CUSPARSE	CULA MAGMA	Thrust NPP	VSIPL SVM OpenCurrent	PhysX OptiX	iray	MATLAB Mathematica
---------------------------------------	---------------	---------------	-----------------------------	----------------	------	-----------------------

Programming Languages

C	C++	Fortran	Java Python Wrappers	DirectCompute	Directives (e.g. OpenACC)
---	-----	---------	----------------------------	---------------	------------------------------



CUDA-Enabled NVIDIA GPUs

<u>Kepler Architecture</u> (compute capabilities 3.x)	GeForce 600 Series	<u>Quadro Kepler Series</u>	Tesla K20 Tesla K10
<u>Fermi Architecture</u> (compute capabilities 2.x)	GeForce 500 Series GeForce 400 Series	<u>Quadro Fermi Series</u>	Tesla 20 Series
<u>Tesla Architecture</u> (compute capabilities 1.x)	<u>GeForce 200 Series</u> <u>GeForce 9 Series</u> <u>GeForce 8 Series</u>	<u>Quadro FX Series</u> <u>Quadro Plex Series</u> <u>Quadro NVS Series</u>	Tesla 10 Series

Img. Src Nvidia



Acknowledgments

- These slides are inspired and partly adapted from
 - Mary Hall (Univ. of Utah)
 - Programming Massively Parallel Processors: A Hands On Approach, available from *http: David Kirk and Wen–mei Hwu, February 2023, Morgan Kaufmann Publishers, ISBN 0123814723.*
 - NVidia, *CUDA Programming Guide*, available from [Docs.nvidia.com/cuda/cuda-c-programming-guide](https://docs.nvidia.com/cuda/cuda-c-programming-guide)